1 Лекция 03. Kernelization: безопасные правила и квадратичное ядро

1.1 Зачем выделять ядро задачи

Ветвление из предыдущей главы ограничивает число вариантов решения функцией бюджета, но каждый рекурсивный вызов всё ещё может обрабатывать большой граф. Другая идея — сначала за полиномиальное время убрать всё, что можно решить без перебора, и оставить маленький эквивалентный экземпляр. **Ядро (kernel)** — такой остаток с доказанной границей размера, зависящей только от параметра. Ядеризация (kernelization) — алгоритм его получения.

Главный пример опять **Vertex Cover**: по графу G и бюджету k спрашиваем, существует ли множество не более k вершин, содержащее хотя бы один конец каждого ребра. Обозначаем число вершин через n, число рёбер через m. $G - v$ удаляет вершину и все её рёбра; степень $\deg(v)$ — число её соседей. Минимальный размер покрытия обозначается $\tau(G)$.

Наша цель — построить ядро с $O(k^2)$ вершинами и рёбрами и доказать связь существования ядра с FPT. Основная идея такова: слишком большая степень заставляет взять вершину; после удаления всех таких вершин малое покрытие не сможет обслужить слишком много оставшихся рёбер.

Безопасное правило (safe reduction rule) заменяет один вход другим с тем же ответом YES/NO. Обычно решения не совпадают буквально: выбранные при предобработке вершины нужно вернуть при восстановлении. Для каждого правила отдельно доказываем сохранение ответов в обе стороны. Полиномиальность правил и малая величина остатка — разные требования: ни одно не следует автоматически из другого.

1.2 Что требуется от предобработки

Ядеризация (kernelization) за полиномиальное время по N преобразует (x, k) в (x', k') той же задачи так, что ответы совпадают, а $|x'| + k' \le g(k)$ для вычислимой g. Если g полиномиальна, ядро полиномиальное. Уменьшение среднего размера реальных входов само по себе не доказывает эту гарантию.

Размер в определении — размер кодировки. «Ядро на $2k$ вершинах» ещё может содержать $\Theta(k^2)$ рёбер. Взвешенная задача требует также контроля числа бит весов. Разрешено вернуть фиксированный YES- или NO-экземпляр. Для Vertex Cover подойдут пустой граф с бюджетом 0 и одно ребро с бюджетом 0.

1.3 Правила для Vertex Cover

Поддерживаем текущий бюджет ℓ, изначально k.

1. Если $\ell < 0$, вернуть NO.
2. Изолированную вершину удалить без изменения ℓ.
3. Если $\deg(v) > \ell$, удалить v и уменьшить ℓ на 1; записать v в обязательную часть ответа.
4. Повторять, пока применимо. Затем, если $m > \ell^2$ или $n > \ell^2 + \ell$, вернуть NO; иначе вернуть (G, ℓ).

Правило 3 безопасно: если покрытие размера не более ℓ не берёт v, оно обязано взять всех её более чем ℓ соседей. Следовательно, все допустимые покрытия берут v. Удаление v и вычитание единицы из бюджета сохраняют ответ в обе стороны. Для правила 2 удаляем

из покрытия изолированную вершину, если она была взята; наоборот, покрытие остатка уже покрывает исходный граф.

После каждого изменения бюджет берётся **текущий**, а не первоначальный. Каждое нетерминальное правило уменьшает число вершин, поэтому не более n итераций. Даже полный пересчёт степеней на каждой итерации полиномиален.

1.4 Почему остаток мал

Пусть остаток имеет покрытие S, $|S| \le \ell$. Максимальная степень не больше ℓ, значит, все вершины S вместе покрывают не более $\ell|S| \le \ell^2$ рёбер. Двойной учёт рёбер внутри S только увеличивает эту верхнюю оценку.

В $V \setminus S$ нет рёбер. Изолированных вершин тоже нет, поэтому каждая вершина вне S имеет соседа в S. Следовательно,

$$|V \setminus S| \le \sum_{v \in S} \deg(v) \le \ell^2, \qquad |V| \le \ell^2 + \ell.$$

Итак, нарушение любой границы доказывает NO. Иначе после перенумерации вершин список рёбер занимает $O((k+1)^2 \log(k+2))$ бит. Ядро остаётся экземпляром Vertex Cover; восстановить покрытие можно, добавив записанные обязательные вершины.

Пример. Звезда с 8 листьями, бюджет 2: центр обязателен, после его удаления все листья изолированы, остаток пуст и бюджет 1. Исходное решение — центр. Но правило «при степени не меньше бюджета вершина обязательна» неверно: в одном ребре при бюджете 1 можно выбрать любой конец.

Поправка к источнику. В VC.3 на с. 22 PDF напечатано соединение условий через `and`, которое не годится для проверки отрицательного бюджета. Здесь $\ell < 0$ проверяется отдельно и безусловно; размерные тесты применяются к неотрицательному бюджету после исчерпания правил.

1.5 Теорема: FPT тогда и только тогда, когда есть ядро

Для разрешимой параметризованной задачи существование ядра эквивалентно FPT; полиномиальное ядро из этого не следует.

Пусть есть алгоритм времени $f(k)N^c$. Симулируем его не более N^{c+1} шагов. Если завершился — выдаём фиксированный экземпляр с известным ответом. Для задач, у которых все входы имеют один и тот же ответ, достаточно любого фиксированного экземпляра; остальные имеют оба вида фиксированных представителей. Если нет — возвращаем вход. Во втором случае $f(k)N^c > N^{c+1}$, значит, $N < f(k)$. Получено ядро, а время предобработки полиномиально: мы не вычисляли $f(k)$, только ограничивали симуляцию.

Обратно, после ядеризации применим к выходу любой останавливающийся решатель. На конечном множестве строк длины не больше $g(k)$ максимальное время такого решателя конечно и вычислимо: можно запустить его на всех строках и дождаться всех завершений. Это даёт функцию $h(k)$ и время $N^{O(1)} + h(k)$. Условие разрешимости здесь существенно.

1.6 Как использовать полученный остаток

Рассмотрим непересекающееся объединение звезды с 8 листьями и треугольника при $k = 3$. Центр звезды имеет степень 8, поэтому выбирается обязательно. Текущий бюджет становится 2; листья после удаления центра исчезают по правилу изоляции. Остаётся треугольник:

степени равны 2, больше текущего бюджета нет, а 3 ребра укладываются в границу 2^2. Предобработка не обязана решить этот остаток. Перебрав его подмножества, выбираем две вершины треугольника и возвращаем также центр: всего 3.

Если исходный бюджет был 2, после центра осталось бы 1. В треугольнике степень 2 больше бюджета: выбираем одну вершину, получаем ребро и бюджет 0; его конец тоже вынужден, после чего бюджет отрицателен. Значит, NO. Использование старого бюджета вместо текущего потеряло бы эти вынужденные шаги.

После ядра можно применить любой точный решатель. Перебор всех подмножеств $O(k^2)$ вершин даёт $2^{O(k^2)}k^{O(1)}$ плюс полиномиальную предобработку. Более выгодно применить ветвление по бюджету: $O^*(1.4656^k)$ на малом остатке. Ядро и ветвление решают разные части работы и естественно сочетаются.

1.7 Самопроверка

Почему запуск 2^kN^2-алгоритма до конца не считается полиномиальной ядеризацией? Почему фиксированные ответы допустимы? Где в оценке размера использовано отсутствие изолированных вершин?

1.8 Практика

Условия задач 03.1–03.5. Решения открывайте после своей попытки.

1.9 Источники и дополнительное чтение

Письменный источник: Cygan et al., *Parameterized Algorithms*, авторская копия 30.05.2016: §2.1, с. 18–20; §2.2.1, с. 21–22. PDF. Номера страниц печатные; номер страницы PDF больше на 16.

Изложение, поясняющие примеры и упражнения подготовлены для этого курса. Ссылки фиксируют происхождение результатов; для чтения конспекта видео не требуется. Необязательные видео собраны в программе.